

PROJECT REPORT

College Event Management System

NAME- SHAGUN SHARMA

REG NO. – 25BAI10074

1)INTRODUCTION

The College Event Management System is a console application built with Java to manage college events neatly. The College Event Management System lets the user add events view all available events and search for a particular event by its name.

The College Event Management System is developed using Java concepts such, as classes, objects, constructors, encapsulation, ArrayList, loops, conditional statements, switch-case, methods and Scanner input. The College Event Management System does not use a database or JDBC so the event information is stored temporarily in an ArrayList while the program is running.

The main purpose of the College Event Management System is to show how Java programming concepts can be combined to create a small and useful real-world application.

2)PROBLEM STATEMENT

It can be hard to handle college events by hand when there are events with different names, different places, different dates and different numbers of people allowed. Keeping this information on paper or in records can make it hard to find and see event details.

So a simple College Event Management System is needed that can:

Add college events.

Keep event information while the program is running.

Show all the events that're available.

Look for an event, by its name.

Give a menu-based way to use it.

Make it less necessary to manage event records by hand.

3) FUNCTIONAL REQUIREMENTS

Functional requirements explain what the system needs to do.

- Add Event

The system lets the user type in:

Event ID

Event Name

Venue

Date

Capacity

The details the user types in are used to make an Event object and are put into the ArrayList.

- View Events

The system shows all events that're, in the ArrayList.

If there are no events the system shows:

"No events found."

- Search Event

The user can type in an event name to look for an event.

The program uses:

`equalsIgnoreCase()`

so that the search works the same no matter if the letters are big or small.

- Exit

The user can choose 4. Exit to stop the program.

- Menu-Based Interface

The main program shows:

College Event Management

1. Add Event

2. View Events

3. Search Event

4. Exit

The user picks an option by using the keyboard.

4)NON- FUNCTIONAL REQUIREMENTS

The system should meet the following -functional requirements:

- Usability

The application has a simple menu-based interface. This makes it easy for someone who is just starting to use the system to get without confusion.

- Performance

The system handles tasks like adding events viewing them and searching through them quickly. This works well when the number of events is small.

- Reliability

The program uses Java methods and logical conditions to carry out the needed operations. This helps ensure dependable behavior.

- Maintainability

The code is split into classes such, as Event, EventManager and Main. This structure makes the code easier to read, understand and change when needed.

- Portability

The application can be run on any computer that has a Java Development Kit (JDK). This means it works across systems without requiring major changes.

5)SYSTEM ARCHITECTURE

The project follows a three-class structure.

COLLEGE EVENT MANAGEMENT SYSTEM

Main.java EventManager.java

User Input/Menu Add / View / Search

Event.java

ID Name Venue

||

Date Capacity Event Details

- Event.java

The Event class represents an individual college event.

Event contains:

```
private int id;  
String name;  
private String venue;  
private String date;  
private int capacity;
```

Event also contains a constructor and getter methods that work with the event information.

- EventManager.java

The EventManager class manages event objects.

EventManager uses:

```
ArrayList<Event> events = ArrayList<>();
```

EventManager contains methods such as:

```
addEvent()  
showEvents()  
searchEvent()
```

- Main.java

The Main class contains the main() method and acts, as the starting point of the application.

Main:

Creates a Scanner object.

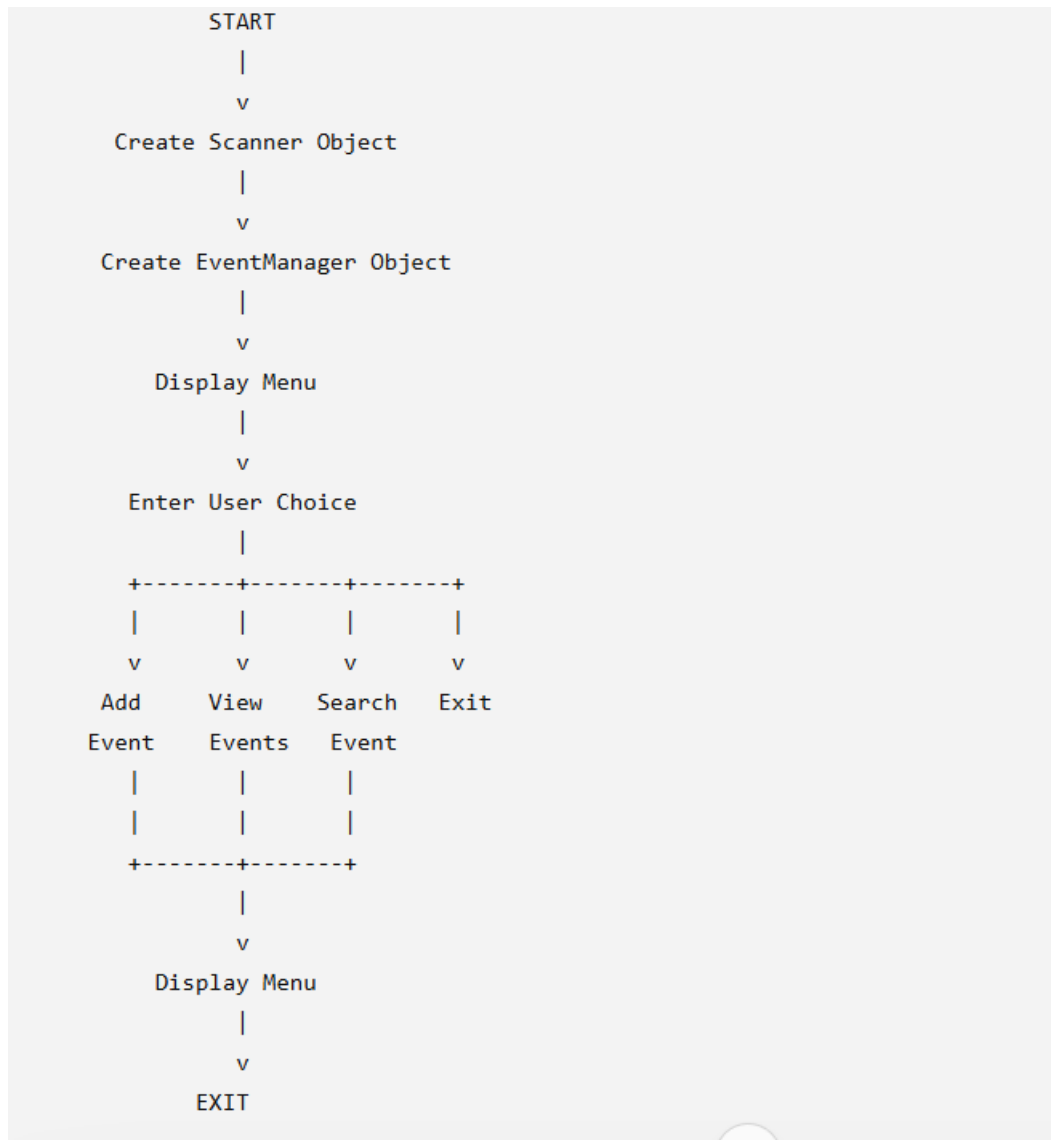
Creates an EventManager object.

Displays the menu.

Accepts user choices.

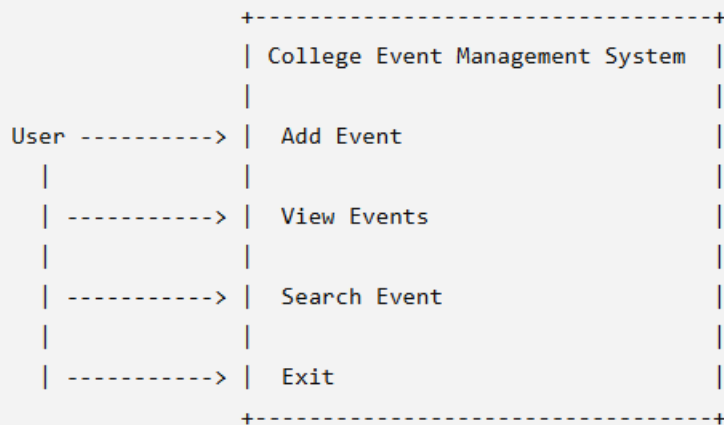
Calls the required methods.

6)WORKFLOW DIAGRAM

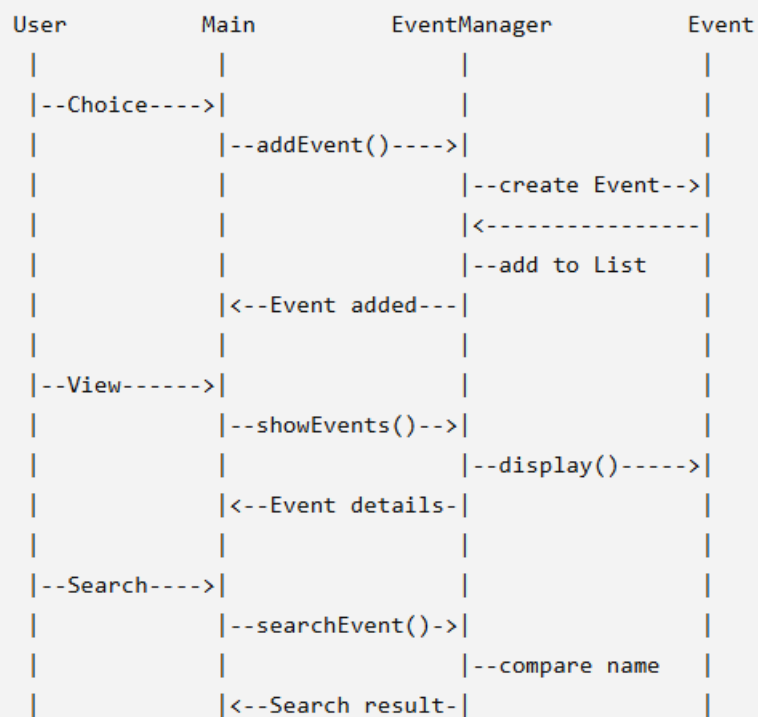


When the program starts, it creates the required objects. The menu is displayed and the user selects an option. According to the selected option, the program adds, displays, or searches events. After completing the operation, the menu is displayed again until the user chooses Exit.

6) DESIGN DIAGRAM



8)SEQUENCE DIAGRAM



9) DESIGN DECISION

I designed the College Event Management System using an object-oriented approach so that the program is easy to understand, use and maintain. The project is divided into three classes: Event, EventManager and Main. The Event class stores the details of an event such as ID, name, venue, date and capacity while the EventManager class handles operations like adding, displaying and searching events. An ArrayList is used to store event objects because the number of events can change during program execution. The variables in the Event class are kept private to provide encapsulation and getter methods are used to access the information. A menu-driven interface, with switch-case is used in the class to make the system simple and user-friendly. The project does not use a database or JDBC because the main aim is to demonstrate Java and OOP concepts while keeping the application simple.

10) Java Concepts Used Putting this together

• Classes and objects • Constructors • Encapsulation • Methods • ArrayList • Scanner •
Conditional statements • Loops • Switch statement

11) TECHNOLOGIES USED

Technology Purpose

Java Main language used for coding

JDK Compiling and running the program

Scanner Getting input, from the user

ArrayList Storing event objects

OOP Keeping the project in classes

12) CHALLENGES FACED

During the development of the project, some challenges were faced.

- **Understanding Classes and Objects**

Initially, understanding how the Event object interacts with EventManager was a challenge. This was solved by separating the event data from the event management operations.

- **Working with ArrayList**

Understanding how to store multiple Event objects in an ArrayList required practice. For example:

```
ArrayList<Event> events = new ArrayList<>();  
The add() method is then used to store objects.
```

- **Taking Different Types of Input**

The program takes both integer and String inputs. Scanner methods such as nextInt() and nextLine() were used carefully to handle these inputs.

- **Searching Events**

The search function required comparing the entered event name with stored event names. equalsIgnoreCase() was used to make the search case-insensitive.

- **Organizing the Code**

Dividing the project into Event, EventManager, and Main classes helped keep the code organized.

13) CONCLUSION

The College Event Management System is a Java console application created to handle basic college event details. The system includes features for adding, looking at and finding events through a menu-based interface.

This project shows Java ideas such as Object-Oriented Programming, classes, objects, constructors, encapsulation, ArrayList, methods, loops if statements, switch-case and taking user input with Scanner.

Even though the current version does not use a database and only keeps data while the program is running it gives a base, for building a bigger event management application later.

Future Improvements

The project can be made better by adding:

Database connection using JDBC

Functions to update and remove events

Student sign-up for events

Login system

Checking how many people can attend events

Storing events

Graphical user interface using Java Swing or JavaFX

Managing event registration and people attending

In total the project shows well how to create a basic real-life management system using fundamental Java programming ideas.